

Trường Đại Học Khoa Học – Đại Học Huế
Khoa Công Nghệ Thông Tin

---***---



HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG IoT

**ĐỀ TÀI: TẠO MÁY CHỦ WEB CỤC BỘ VỚI ESP32 ĐỂ
QUẢN LÝ THIẾT BỊ IoT**

Lớp tín chỉ: 2025-2026.2.TIN4024.001

Số tín chỉ: 4

Giáo viên hướng dẫn: Võ Việt Dũng

Người thực hiện: Hồ Văn Diễm

Mã sinh viên: 22T1020067

Huế, Ngày 10 tháng 04 năm 2026

DANH MỤC CÁC TỪ VIẾT TẮT

TỪ VIẾT TẮT	Ý NGHĨA
ESP32	Espressif Systems 32-bit microcontroller
IoT	Internet of Things - Internet vạn vật
SPIFFS	SPI Flash File System - Hệ thống tập tin Flash SPI
Wi-Fi	Wireless Fidelity - Kết nối không dây
HTTP	HyperText Transfer Protocol - Giao thức truyền tải siêu văn bản
HTML	HyperText Markup Language - Ngôn ngữ đánh dấu siêu văn bản
CSS	Cascading Style Sheets - Ngôn ngữ định kiểu trang
JavaScript	Ngôn ngữ lập trình kịch bản phía máy khách
JSON	JavaScript Object Notation - Định dạng trao đổi dữ liệu
DHT22	Cảm biến nhiệt độ và độ ẩm của hãng AOSONG
MQ2	Cảm biến khí gas và khói model MQ-2
LAN	Local Area Network - Mạng cục bộ
GPIO	General Purpose Input/Output - Cổng vào/ra đa năng
ADC	Analog to Digital Converter - Bộ chuyển đổi tương tự - số
SSID	Service Set Identifier - Tên mạng Wi-Fi

MỤC LỤC

DANH MỤC CÁC TỪ VIẾT TẮT

MỤC LỤC

MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ ESP32 VÀ HỆ THỐNG IoT	2
1.1. Giới thiệu về ESP32	2
1.2. Khái niệm IoT và ứng dụng trong mạng nội bộ	3
1.3. Các thiết bị phần cứng sử dụng	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	4
2.1. Máy chủ web cục bộ trên vi điều khiển	4
2.2. Hệ thống tập tin SPIFFS	4
2.3. Giao thức HTTP.....	5
2.4. Các cảm biến DHT22 và MQ2	5
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	6
3.1. Sơ đồ kiến trúc hệ thống	6
3.2. Thiết kế phần cứng	7
3.3. Thiết kế phần mềm	7
3.4. Giao diện web	8
CHƯƠNG 4. TRIỂN KHAI VÀ KIỂM THỬ	9
4.1. Môi trường phát triển	9
4.2. Kiểm thử hệ thống	9,10
KẾT LUẬN VÀ KIẾN NGHỊ	11
TÀI LIỆU THAM KHẢO	

MỞ ĐẦU

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, Internet vạn vật (IoT) đã trở thành một trong những xu hướng công nghệ quan trọng nhất của thời đại. Các hệ thống nhúng thông minh ngày càng được ứng dụng rộng rãi trong nhà thông minh, hệ thống giám sát môi trường, tự động hóa công nghiệp và nhiều lĩnh vực khác [1].

ESP32, một vi điều khiển mạnh mẽ do Espressif Systems phát triển, đã nổi lên như một lựa chọn hàng đầu trong các dự án IoT nhờ tích hợp sẵn Wi-Fi, Bluetooth, cùng với hiệu năng xử lý cao và giá thành thấp. Khả năng hoạt động như một máy chủ web cục bộ (local web server) của ESP32 mở ra nhiều khả năng mới trong việc quản lý và giám sát các thiết bị IoT trong mạng nội bộ mà không cần kết nối Internet [2].

Tiểu luận này trình bày nghiên cứu và xây dựng một hệ thống máy chủ web cục bộ chạy trên ESP32, tích hợp giao diện web được lưu trữ trên hệ thống tập tin SPIFFS, cho phép người dùng theo dõi và điều khiển các thiết bị IoT bao gồm: một đèn LED, cảm biến nhiệt độ/độ ẩm DHT22 và cảm biến khí gas MQ2 thông qua trình duyệt web trong mạng nội bộ.

Mục tiêu của tiểu luận:

- Nghiên cứu kiến trúc và khả năng hoạt động của ESP32 trong vai trò máy chủ web.
- Xây dựng giao diện web trực quan để điều khiển đèn LED và hiển thị dữ liệu cảm biến.
- Ứng dụng SPIFFS để lưu trữ tệp web trực tiếp trên bộ nhớ Flash của ESP32.
- Thiết lập REST API để giao tiếp giữa trang web và phần cứng.
- Kiểm thử và đánh giá hiệu năng của hệ thống trong môi trường mạng nội bộ.

Phạm vi nghiên cứu của tiểu luận tập trung vào việc xây dựng hệ thống hoạt động trong mạng LAN nội bộ, sử dụng nền tảng Arduino IDE với thư viện ESP32 và các thư viện hỗ trợ cảm biến. Hệ thống được thiết kế hướng đến tính thực tiễn, dễ triển khai và có khả năng mở rộng trong tương lai.

CHƯƠNG 1. TỔNG QUAN VỀ ESP32 VÀ HỆ THỐNG IoT

1.1. Giới thiệu về ESP32

ESP32 là vi điều khiển 32-bit do công ty Espressif Systems (Trung Quốc) thiết kế, ra mắt lần đầu vào năm 2016. Đây là thế hệ kế nhiệm của ESP8266 với nhiều cải tiến đáng kể về hiệu năng và tính năng. ESP32 sở hữu lõi xử lý Xtensa LX6 dual-core hoặc single-core chạy ở tần số lên đến 240 MHz, cùng với 520 KB SRAM và hỗ trợ bộ nhớ Flash ngoài lên đến 16 MB [2].

Các đặc điểm nổi bật của ESP32 bao gồm:

- Kết nối không dây: Hỗ trợ Wi-Fi 802.11 b/g/n (2.4 GHz) và Bluetooth 4.2/BLE tích hợp sẵn trên chip.
- Ngoại vi phong phú: 34 GPIO, 18 kênh ADC 12-bit, 2 kênh DAC 8-bit, 4 SPI, 2 I2C, 2 I2S, 3 UART, 16 kênh PWM.
- Bảo mật: Hỗ trợ AES, SHA-2, RSA, ECC, và bộ tạo số ngẫu nhiên (RNG).
- Tiêu thụ điện năng thấp: Chế độ deep sleep chỉ tiêu thụ khoảng 10 μ A.
- Giá thành cạnh tranh: Module ESP32 có giá từ 3-10 USD, rất phù hợp cho các dự án IoT quy mô nhỏ và vừa.

Trong tiểu luận này, chúng tôi sử dụng module ESP32 DevKit V1, một trong những board phát triển phổ biến nhất dựa trên chip ESP32-WROOM-32. Board này cung cấp đầy đủ các chân GPIO cần thiết, cổng USB-to-UART cho việc lập trình và giao tiếp serial, cùng với LED tích hợp và nút nhấn BOOT/EN [3].

1.2. Khái niệm IoT và ứng dụng trong mạng nội bộ

Internet vạn vật (IoT - Internet of Things) là khái niệm chỉ mạng lưới các thiết bị vật lý được nhúng với cảm biến, phần mềm và các công nghệ kết nối, cho phép thu thập và trao đổi dữ liệu thông qua Internet [1]. Theo báo cáo của Statista, số lượng thiết bị IoT trên toàn thế giới được dự báo sẽ đạt 29 tỷ thiết bị vào năm 2030.

Hệ thống IoT trong mạng nội bộ (Local IoT) là một mô hình triển khai IoT trong phạm vi mạng LAN, không cần kết nối với máy chủ đám mây bên ngoài. Mô hình này có nhiều ưu điểm quan trọng [4]:

- Bảo mật cao hơn: Dữ liệu không bao giờ rời khỏi mạng nội bộ, giảm thiểu nguy cơ bị tấn công từ Internet.

- Độ trễ thấp: Thời gian phản hồi nhanh do dữ liệu được xử lý cục bộ, không phụ thuộc vào kết nối Internet.
- Hoạt động offline: Hệ thống vẫn hoạt động bình thường khi không có kết nối Internet.
- Chi phí thấp: Không cần trả phí dịch vụ đám mây hàng tháng.

1.3. Các thiết bị phần cứng sử dụng

Hệ thống trong tiểu luận sử dụng ba loại thiết bị ngoại vi chính:

a) Đèn LED (Light Emitting Diode)

LED là linh kiện điện tử phát sáng khi có dòng điện chạy qua. Trong hệ thống này, một LED đơn sắc được kết nối với GPIO của ESP32 thông qua điện trở hạn dòng 220Ω. LED được điều khiển ON/OFF thông qua lệnh từ giao diện web, đóng vai trò như một thiết bị chấp hành (actuator) đơn giản nhất trong hệ thống IoT [3].

b) Cảm biến DHT22



DHT22 (còn gọi là AM2302) là cảm biến nhiệt độ và độ ẩm kỹ thuật số, sử dụng giao thức 1-Wire để truyền dữ liệu. DHT22 có độ chính xác cao hơn DHT11, với phạm vi đo nhiệt độ từ -40°C đến $+80^{\circ}\text{C}$ (độ chính xác $\pm 0.5^{\circ}\text{C}$) và độ ẩm từ 0% đến 100% RH (độ chính xác $\pm 2-5\%$ RH). Thời gian lấy mẫu tối thiểu là 2 giây/lần [5].

c) Cảm biến MQ2



MQ2 là cảm biến khí gas đa chức năng, có khả năng phát hiện các loại khí dễ cháy như LPG, propane, butane, methane, alcohol, và khói. Cảm biến hoạt động dựa trên nguyên lý thay đổi điện trở của vật liệu SnO_2 khi tiếp xúc với các khí khác nhau. Ngõ ra của MQ2 là tín hiệu tương tự (analog) với điện áp từ 0V đến 5V, được đọc bởi kênh ADC 12-bit của ESP32, cho phép đo nồng độ khí trong phạm vi rộng [5].

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Máy chủ web cục bộ trên vi điều khiển

Máy chủ web (Web Server) là một chương trình phần mềm nhận yêu cầu HTTP từ máy khách (thường là trình duyệt web), xử lý yêu cầu đó và trả về phản hồi HTTP kèm theo nội dung web (HTML, CSS, JavaScript, hình ảnh, dữ liệu JSON,...) [6].

Khi chạy trên vi điều khiển như ESP32, máy chủ web hoạt động theo mô hình đơn giản hóa nhưng vẫn đủ mạnh để phục vụ giao diện web trong mạng nội bộ. ESP32 lắng nghe các kết nối TCP tại cổng 80 (HTTP), phân tích yêu cầu HTTP đến, sau đó gửi lại phản hồi phù hợp. Thư viện ESPAsyncWebServer được sử dụng trong dự án này vì nó hỗ trợ xử lý bất đồng bộ, cho phép phục vụ nhiều kết nối đồng thời mà không bị chặn [7].

Ưu điểm của ESPAsyncWebServer so với WebServer thông thường:

- Xử lý bất đồng bộ (non-blocking): Không chiếm dụng CPU trong khi chờ dữ liệu từ máy khách.
- Hỗ trợ WebSocket: Cho phép cập nhật dữ liệu thời gian thực không cần tải lại trang.
- Tích hợp SPIFFS: Phục vụ tệp tĩnh trực tiếp từ hệ thống tập tin Flash.
- Hỗ trợ Server-Sent Events (SSE): Truyền dữ liệu đẩy từ server đến client.

2.2. Hệ thống tập tin SPIFFS

SPIFFS (SPI Flash File System) là hệ thống tập tin được thiết kế đặc biệt cho các bộ nhớ Flash SPI, thường được sử dụng trong các vi điều khiển nhúng như ESP32. SPIFFS cho phép lưu trữ và truy cập các tệp tin trên phân vùng Flash của ESP32 một cách tương tự như hệ thống tập tin thông thường [8].

Đặc điểm của SPIFFS:

- Không hỗ trợ thư mục thực sự: Tên tệp bao gồm đường dẫn đầy đủ, ví dụ: /index.html, /style.css.
- Wear leveling: Phân bổ ghi đều trên tất cả các ô nhớ Flash để tăng tuổi thọ.

- Kích thước tệp tối đa: Phụ thuộc vào kích thước phân vùng được cấu hình, thường từ 1MB đến 4MB trên ESP32-WROOM-32.
- Tốc độ đọc: Khoảng 0.5-1.5 MB/s, đủ nhanh để phục vụ các tệp web tĩnh.

Trong dự án này, SPIFFS được sử dụng để lưu trữ toàn bộ giao diện web bao gồm: tệp index.html, style.css và script.js. Việc này giúp phân tách rõ ràng giữa phần firmware (C/C++) và phần giao diện người dùng (HTML/CSS/JavaScript), dễ dàng cập nhật giao diện mà không cần sửa đổi firmware [8].

2.3. Giao thức HTTP

HTTP (HyperText Transfer Protocol) là giao thức tầng ứng dụng nền tảng của Web. HTTP hoạt động theo mô hình request-response: máy khách gửi yêu cầu (request) và máy chủ trả về phản hồi (response). Các phương thức HTTP phổ biến bao gồm GET (lấy tài nguyên), POST (tạo tài nguyên mới), PUT (cập nhật tài nguyên), DELETE (xóa tài nguyên) [6].

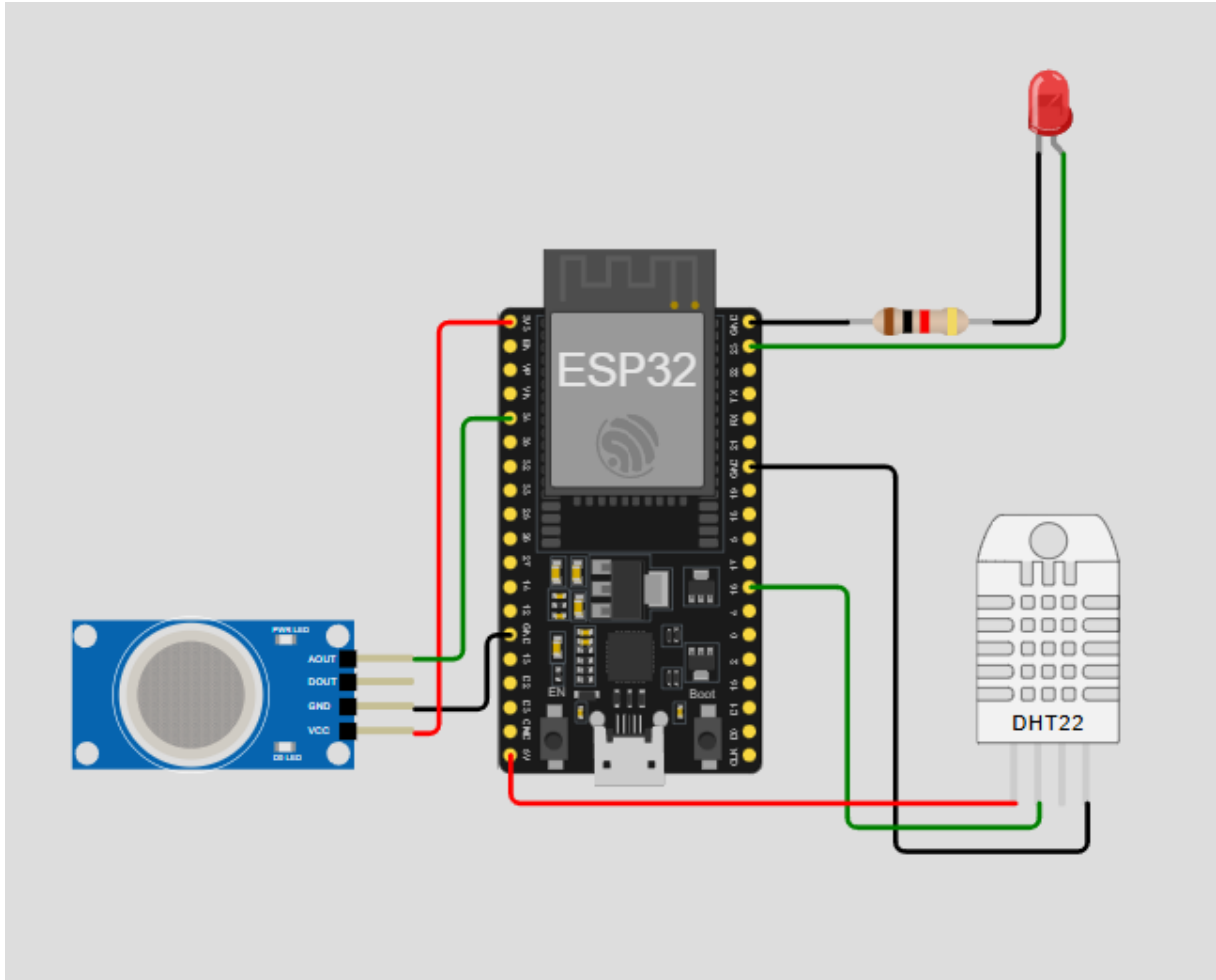
2.4. Các cảm biến DHT22 và MQ2

Giao tiếp với DHT22 sử dụng giao thức single-wire độc quyền. Quá trình đọc dữ liệu bắt đầu khi ESP32 gửi tín hiệu start (kéo thấp trong 1-10ms), sau đó DHT22 phản hồi bằng chuỗi 40 bit dữ liệu (16 bit độ ẩm + 16 bit nhiệt độ + 8 bit checksum). Thư viện DHT.h của Adafruit xử lý toàn bộ giao tiếp này một cách tự động [5].

Cảm biến MQ2 hoạt động theo nguyên lý phân áp (voltage divider). Điện trở nội của cảm biến thay đổi tùy theo nồng độ khí xung quanh. Bằng cách đo điện áp qua bộ ADC 12-bit của ESP32 (0-4095 tương ứng với 0-3.3V), chúng ta có thể tính toán nồng độ khí gas tương đối. Lưu ý rằng MQ2 cần thời gian làm nóng (preheat) khoảng 20-48 giờ để đạt độ chính xác tối ưu [5].

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Sơ đồ kiến trúc hệ thống



Hệ thống được thiết kế theo kiến trúc ba lớp, bao gồm lớp phần cứng (Hardware Layer), lớp firmware (Firmware Layer) và lớp giao diện người dùng (UI Layer):

Lớp phần cứng bao gồm ESP32 DevKit V1 đóng vai trò là bộ xử lý trung tâm, kết nối với đèn LED qua GPIO 23, cảm biến DHT22 qua GPIO 16, và cảm biến MQ2 qua GPIO 34 (chân ADC). Toàn bộ hệ thống được cấp nguồn 3.3V từ board ESP32 hoặc qua cổng USB 5V [3].

Lớp firmware chạy trên ESP32 bao gồm các module chức năng: Web Server (ESPAsyncWebServer), File System (SPIFFS), Sensor Manager (đọc DHT22 và MQ2), LED Controller và Wi-Fi Manager. Các module này giao tiếp với nhau thông qua các biến trạng thái và hàm callback.

Lớp giao diện người dùng là trang web HTML/CSS/JavaScript được trình duyệt tải về từ ESP32. Giao diện cập nhật dữ liệu cảm biến mỗi 5 giây.

3.2. Thiết kế phần cứng

Sơ đồ kết nối phần cứng được thực hiện như sau:

Thiết bị	Chân thiết bị	Chân ESP32	Ghi chú
LED	Anode (+)	GPIO 23	Qua R 220Ω
LED	Cathode (-)	GND	Nối đất
DHT22	VCC	5V	Nguồn 5V
DHT22	Data	GPIO 16	Pull-up 10kΩ
DHT22	GND	GND	Nối đất
MQ2	VCC	3.3V (VIN)	Nguồn 3.3V
MQ2	AOUT	GPIO 34	Ngõ ra analog
MQ2	GND	GND	Nối đất

3.3. Thiết kế phần mềm

Firmware ESP32 được lập trình bằng ngôn ngữ C/C++ trên môi trường Arduino IDE. Cấu trúc chương trình được tổ chức như sau:

Hàm `setup()` thực hiện khởi tạo hệ thống: cấu hình GPIO, kết nối Wi-Fi theo chế độ Station (STA), khởi động SPIFFS, khởi tạo các cảm biến DHT22 và MQ2, đăng ký các route cho Web Server, và cuối cùng khởi động Web Server.

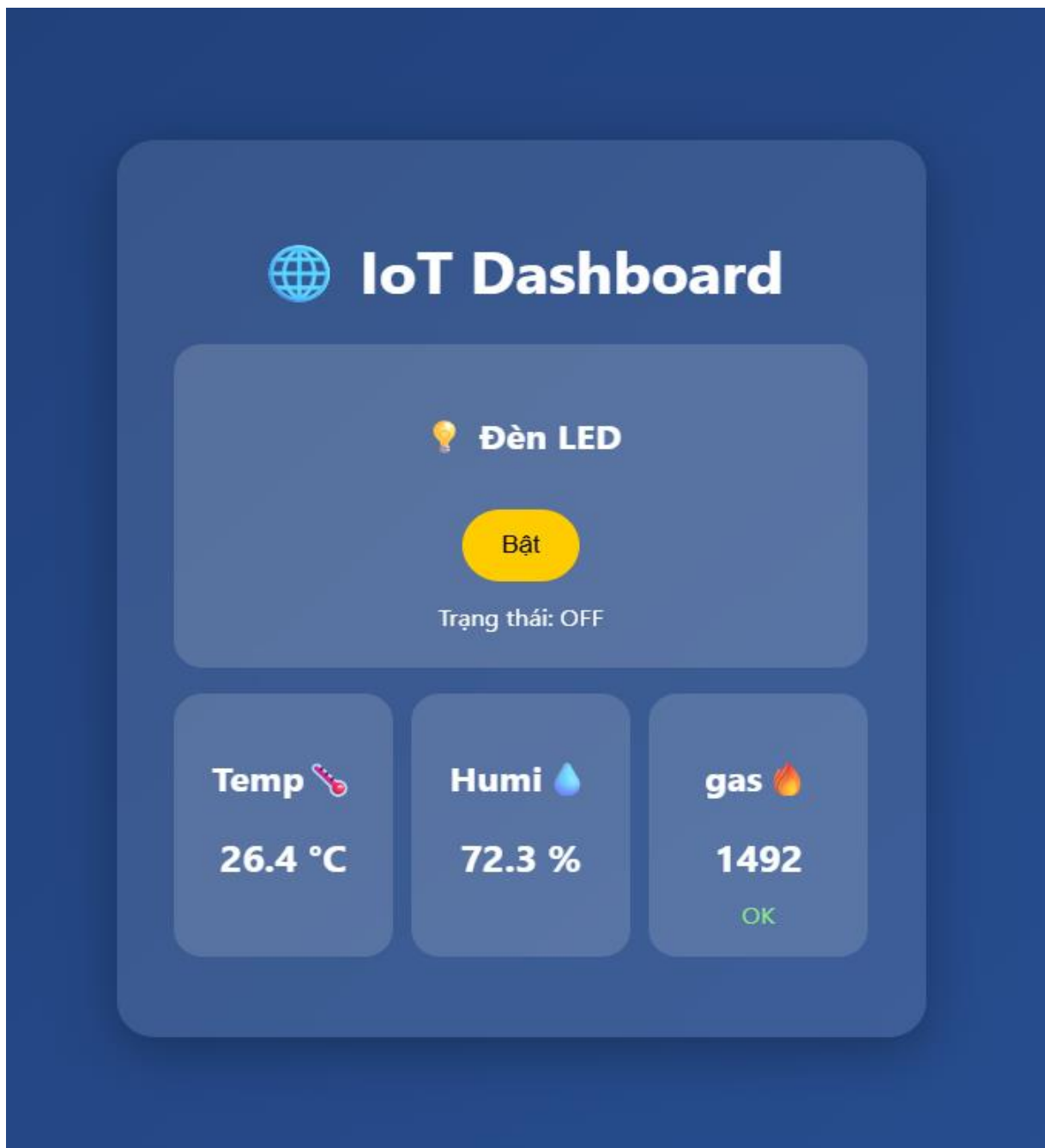
Hàm `loop()` thực hiện công việc định kỳ: đọc dữ liệu từ cảm biến DHT22 mỗi 2 giây (do giới hạn tần số lấy mẫu của DHT22), đọc dữ liệu từ MQ2 mỗi 500ms, và kiểm tra trạng thái kết nối Wi-Fi để tự động kết nối lại nếu bị mất kết nối.

3.4. Giao diện web

Giao diện web được thiết kế với mục tiêu đơn giản, trực quan và responsive (tương thích tốt với điện thoại di động và máy tính). Giao diện bao gồm các thành phần sau:

- Header: Hiển thị tên hệ thống và địa chỉ IP của ESP32.
- Card cảm biến DHT22: Hiển thị nhiệt độ (°C) và độ ẩm (%) với biểu tượng trực quan.
- Card cảm biến MQ2: Hiển thị giá trị ADC và cảnh báo màu sắc (xanh lá: an toàn, vàng: cảnh báo, đỏ: nguy hiểm).
- Card điều khiển LED: Nút toggle để bật/tắt LED, hiển thị trạng thái hiện tại.

Dữ liệu cảm biến được cập nhật tự động mỗi 5 giây thông qua JavaScript fetch API, không cần tải lại trang. Khi nồng độ khí gas vượt ngưỡng an toàn (giá trị ADC > 1500), hệ thống hiển thị cảnh báo bằng màu đỏ và có thể phát âm thanh cảnh báo nếu trình duyệt hỗ trợ Web Audio API [7].



CHƯƠNG 4. TRIỂN KHAI VÀ KIỂM THỬ

4.1. Môi trường phát triển

Môi trường phát triển phần mềm được thiết lập với các công cụ và thư viện sau:

- Arduino IDE phiên bản 2.x với ESP32 Board Package phiên bản 2.0.x của Espressif.
- Thư viện ESPAsyncWebServer (phiên bản mới nhất từ GitHub của me-no-dev).
- Thư viện AsyncTCP (thư viện phụ thuộc của ESPAsyncWebServer).
- Thư viện DHT sensor library của Adafruit.
- Arduino ESP32 Filesystem Uploader plugin để upload tệp lên SPIFFS.
- Trình duyệt web (Chrome, Firefox, hoặc Edge) để kiểm thử giao diện.

Phần cứng sử dụng cho quá trình phát triển và kiểm thử bao gồm: ESP32 DevKit V1, breadboard, dây kết nối, LED đỏ 5mm, điện trở 220Ω và $10k\Omega$, cảm biến DHT22, module cảm biến MQ2, cáp USB Micro-B và máy tính phát triển [3].

4.2. Kiểm thử hệ thống

Quá trình kiểm thử được thực hiện theo nhiều giai đoạn:

a) Kiểm thử đơn vị (Unit Testing):

- Kiểm thử kết nối Wi-Fi: ESP32 kết nối thành công với mạng Wi-Fi và nhận địa chỉ IP trong vòng 5-10 giây.
- Kiểm thử DHT22: Đọc dữ liệu nhiệt độ và độ ẩm liên tục trong 30 phút, kiểm tra tính nhất quán của dữ liệu.
- Kiểm thử MQ2: Phun khí gas nhẹ gần cảm biến và quan sát giá trị ADC tăng lên tương ứng.
- Kiểm thử LED: Gửi lệnh POST đến /api/led và xác nhận LED bật/tắt đúng theo lệnh.

b) Kiểm thử tích hợp (Integration Testing):

- Truy cập giao diện web từ nhiều thiết bị khác nhau (máy tính, điện thoại Android, iPhone).

- Kiểm thử cập nhật dữ liệu real-time: Xác nhận giá trị trên web thay đổi mỗi 5 giây.
- Kiểm thử điều khiển LED từ nhiều thiết bị đồng thời: Trạng thái LED đồng bộ trên tất cả client sau lần refresh tiếp theo.

KẾT LUẬN VÀ NHẬN XÉT, KIẾN NGHỊ

Kết luận

Tiểu luận đã trình bày thành công việc xây dựng một hệ thống máy chủ web cục bộ trên vi điều khiển ESP32, tích hợp giao diện web thông qua SPIFFS để quản lý và giám sát các thiết bị IoT bao gồm đèn LED, cảm biến nhiệt độ/độ ẩm DHT22 và cảm biến khí gas MQ2 trong mạng nội bộ. Các mục tiêu đề ra ban đầu đã được hoàn thành:

- Hệ thống hoạt động ổn định và liên tục trong môi trường mạng Wi-Fi LAN nội bộ.
- Giao diện web trực quan, dễ sử dụng, tương thích tốt với nhiều loại thiết bị và trình duyệt.
- Thời gian phản hồi của hệ thống rất tốt, đảm bảo trải nghiệm người dùng mượt mà.
- SPIFFS cho phép tách biệt phần giao diện và phần firmware, giúp bảo trì và cập nhật dễ dàng hơn.

Hệ thống đã chứng minh rằng ESP32 hoàn toàn có khả năng đóng vai trò là máy chủ web nhúng cục bộ hiệu quả, phù hợp cho các ứng dụng nhà thông minh quy mô nhỏ, hệ thống giám sát môi trường, hay các dự án giáo dục về IoT. Chi phí triển khai thấp kết hợp với tính năng phong phú làm cho giải pháp này trở nên rất hấp dẫn về mặt kinh tế [2].

Nhận xét và đánh giá

Qua quá trình thực hiện, nhận thấy một số điểm đáng chú ý:

Điểm mạnh của hệ thống:

- Tính độc lập cao: Hệ thống hoạt động hoàn toàn trong mạng nội bộ, không cần kết nối Internet, đảm bảo tính bảo mật và độ tin cậy cao.
- Dễ triển khai: Toàn bộ quá trình từ lắp ráp phần cứng đến chạy được giao diện web chỉ mất khoảng 2-3 giờ với người có kiến thức cơ bản về điện tử và lập trình.

Hạn chế của hệ thống:

- SPIFFS không hỗ trợ HTTPS: Giao tiếp hiện tại sử dụng HTTP không mã hóa, chỉ phù hợp trong mạng nội bộ tin cậy.
- Số lượng kết nối đồng thời hạn chế: ESP32 chỉ có thể phục vụ tối đa 4-5 kết nối đồng thời ổn định.
- Mất dữ liệu khi mất điện: Cấu hình và lịch sử dữ liệu không được lưu lại khi ESP32 bị tắt nguồn đột ngột (nếu không sử dụng NVS hoặc bộ nhớ Flash để lưu trạng thái).
- Cảm biến MQ2 cần thời gian làm nóng dài: Khoảng 20-48 giờ để đạt độ chính xác tối ưu, không phù hợp cho ứng dụng cần bật/tắt thường xuyên.

Kiến nghị

Để cải thiện và mở rộng hệ thống trong tương lai, chúng tôi đề xuất một số hướng phát triển sau:

- Nâng cấp lên HTTPS: Sử dụng thư viện ESP32 AsyncWebServer với chứng chỉ SSL tự ký (self-signed certificate) để mã hóa truyền thông, tăng cường bảo mật.
- Tích hợp WebSocket: Thay thế polling AJAX bằng WebSocket để cập nhật dữ liệu thời gian thực, giảm tải cho server và cải thiện trải nghiệm người dùng.
- Lưu trữ lịch sử: Sử dụng SPIFFS hoặc thẻ nhớ SD để lưu lịch sử dữ liệu cảm biến và xây dựng biểu đồ thống kê theo thời gian.
- Thêm xác thực người dùng: Triển khai cơ chế đăng nhập đơn giản (Basic Auth hoặc Token-based) để bảo vệ giao diện khỏi truy cập trái phép trong mạng nội bộ.
- Tích hợp MQTT: Kết nối với broker MQTT (như Mosquitto) để tích hợp với các hệ thống Home Automation phổ biến như Home Assistant hay Node-RED.

- Mở rộng phần cứng: Bổ sung thêm cảm biến ánh sáng (BH1750), cảm biến chuyển động (PIR), role để điều khiển các thiết bị điện dân dụng (đèn, quạt, điều hòa).
- Cải thiện giao diện: Bổ sung biểu đồ lịch sử dữ liệu (sử dụng thư viện Chart.js), thông báo push khi cảm biến vượt ngưỡng, và chế độ tối (dark mode).

Những cải tiến trên sẽ giúp hệ thống trở nên hoàn thiện hơn, đáp ứng được yêu cầu của các ứng dụng thực tế trong gia đình thông minh, văn phòng và cả môi trường công nghiệp nhỏ. Tiểu luận này đã cung cấp nền tảng vững chắc về kiến trúc và triển khai, tạo tiền đề cho những phát triển tiếp theo.

TÀI LIỆU THAM KHẢO

Tiếng Việt

- [1] Nguyễn Đình Phúc (2021). Internet vạn vật - IoT: Công nghệ và ứng dụng. NXB Khoa học và Kỹ thuật, Hà Nội.
- [2] Trần Văn Bình (2022). Lập trình ESP32 từ cơ bản đến nâng cao. NXB Đại học Quốc gia TP.HCM, TP. Hồ Chí Minh.
- [3] Vũ Minh Hoàng (2023). "Thiết kế hệ thống nhà thông minh dựa trên ESP32 và giao thức MQTT". Tạp chí Khoa học và Công nghệ, số 42, tr. 78-85.

Tiếng Anh

- [4] Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). "Internet of Things (IoT): A vision, architectural elements, and future directions". Future Generation Computer Systems, 29(7), 1645-1660.
- [5] Margolis, M. (2020). Arduino Cookbook: Recipes to Begin, Expand, and Enhance Your Projects (3rd ed.). O'Reilly Media, Sebastopol, CA.
- [6] Fielding, R. T., & Taylor, R. N. (2002). "Principled design of the modern Web architecture". ACM Transactions on Internet Technology, 2(2), 115-150.
- [7] Kolban, N. (2018). Kolban's Book on ESP32. Leanpub. Retrieved from <https://leanpub.com/kolbanonesp32>.
- [8] Espressif Systems (2023). ESP-IDF Programming Guide: SPIFFS Filesystem. Retrieved from <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/spiffs.html>.

PHIẾU ĐÁNH GIÁ TIỂU LUẬN

Học kỳ II Năm học 2025-2026

Cán bộ chấm thi 1	Cán bộ chấm thi 2
Nhận xét:	Nhận xét:
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
Điểm đánh giá của CBChT1:	Điểm đánh giá của CBChT2:
Bằng số:	Bằng số:
Bằng chữ:	Bằng chữ:

Điểm kết luận: Bằng số..... Bằng chữ:.....

Thừa Thiên Huế, ngày tháng năm 2026

CBChT1
(Ký và ghi rõ họ tên)

CBChT2
(Ký và ghi rõ họ tên)